Does a structured NLP layer make an agentic appointment-booking chatbot better? A comparison on identical test conversations

A case study in a Swiss-banking scheduling assistant built on a tool-calling LLM agent

Author: Roman Pfeiler
Affiliation: University of Bern, CAS Natural Language Processing
Contact: roman.pfeiler@gmail.com
Code: https://github.com/RomanPfeiler/appointment-scheduling-chatbot
Live Demo: https://huggingface.co/spaces/JosefPilter/appointments-chatbot
Date: June 2026

Confidentiality note. This report describes a demonstration system built for an academic project. It uses a Swiss retail bank as its setting, but no real customer data of any kind is used. The banking framing is a realistic domain for the task, not a claim about any specific institution.

Abstract

An appointment-scheduling chatbot for a Swiss retail bank is built as an agentic system. At its centre is a tool-calling language-model agent (Gemini 2.5 Flash) that talks to a mock booking service through a small set of well-defined tools. The system is used to ask one research question: **does adding a structured NLP layer (datetime parsing and entity extraction) measurably improve the agent's behaviour over a single-prompt baseline that does all the reasoning itself?** Three NLP conditions (a classical rule-based pipeline, a local 3-billion-parameter model, and an API-LLM pipeline) are tested by running an identical, fixed selection of synthetic conversations under every condition — 120 paired conversations per condition (60 scenarios, each run twice), drawn from a ~500-scenario suite — and comparing with paired statistics. The result is a clean, well-powered **null. No NLP condition significantly improves end-to-end booking, turn count, faithfulness, or datetime accuracy.** In isolation the components do differ. The rule-based and local approaches are measurably weaker than the cloud API at reading non-native date phrasings and entity paraphrases. But that API model is the same engine the baseline already runs, so its recognition quality adds nothing new, and a conversation-level audit shows why the rest leaks away. A correct annotation hint duplicates what the agent would have done anyway, while a wrong annotation hint (most often a re-reading of a mid-conversation follow-up with no memory of the rest of the chat) anchors the agent to a wrong window it then has to recover from. A probe with a weaker agent model points the same way. The single significant end-to-end win came from a **deterministic window-expansion policy in the executor** that broadens an empty availability search. Dead-end turns fell 54% and the judge's negotiation and customer-experience ratings rose, at a measured latency cost. A combination run shows the two levers are redundant rather than complementary. The methodological contribution is a two-layer evaluation that keeps "the component works on its own" separate from "the agent actually got better." One layer scores each NLP component in isolation; the other scores the whole conversation on the metrics that matter to a customer — booking completion, number of turns, "nothing available" dead-ends, and a deterministic faithfulness check that catches invented slots with an LLM-as-a-judge adding quality ratings such as negotiation and customer experience.

1. Introduction

1.1 Motivation

Booking an appointment is one of the most common reasons a customer contacts a retail bank, and one of the most frustrating to do in a conversation. The customer has a rough time in mind ("sometime next week, ideally an afternoon"), the bank has a calendar with real constraints, and the two have to be reconciled in a short back-and-forth. It is a high-volume, high-friction interaction. That makes it an honest place to study an agentic conversational architecture. The task is genuinely hard for a dialogue system (it requires understanding time expressions, tracking what the user already said, and negotiating around an unhelpful calendar). A Swiss retail bank serves as the setting because many of its customers are non-native English speakers whose phrasing carries German, French, or Italian habits, so the time expressions the system must parse are more varied than clean American English.

1.2 The task

The system books **one** appointment. To do so it must settle three things with the customer: the **topic** (Investing, Mortgage, or Pension), the **contact medium** through which the appointment takes place (a branch meeting, an online meeting, or a phone call), and a **datetime**. It must then confirm a real, available slot.

> **Illustrative exchange**
>
> A customer opens with "I'd like to talk about my pension, next week Tuesday afternoon, by phone." The agent must recognise the topic (pension) and the contact medium (phone), resolve "next week Tuesday afternoon" against today's date and the bank's calendar, check whether that window is free, and, if it is not, propose a nearby alternative rather than simply replying "nothing available". Each of those steps is a place the conversation can go wrong.

The booking is subject to business rules a real bank would impose. Appointments are 60 minutes long, fall on weekdays inside business hours (08:00–17:00), must be at least 24 hours in the future, and are expressed in a single local timezone (Central European Time). Not every contact medium is offered for every topic (a mortgage has no phone option, a pension no in-branch option). Success is not simply "made a booking": for requests the system cannot satisfy (a recurring series, an out-of-scope request, an empty calendar), the correct outcome is a polite, accurate refusal or "nothing available" message, never a fabricated booking.

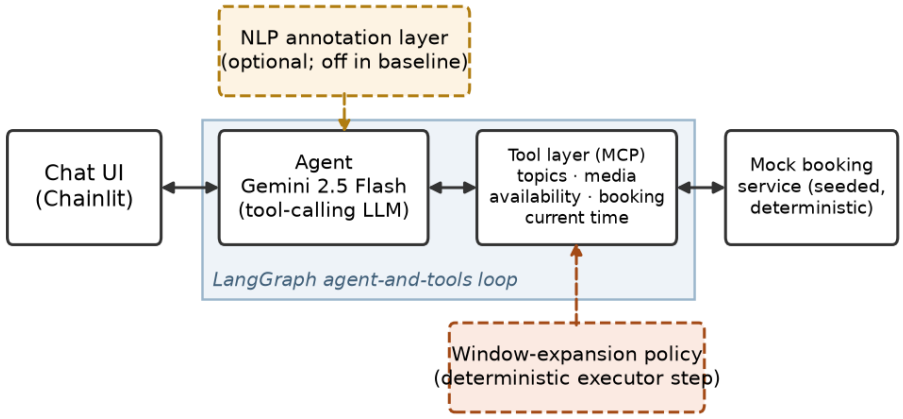


Figure 1. The system as four separable layers: the user interface, the agent (a tool-calling LLM), the deterministic tool layer that fronts a mock booking service, and the optional NLP annotation layer whose value is the subject of this study.

1.3 Approach: the agentic architecture and the research question

The system is built as four loosely coupled layers (Figure 1), so that each can be tested on its own and swapped without disturbing the others:

- a **chat interface** (Chainlit) the customer types into.
- an **agent**, a single tool-calling LLM (Gemini 2.5 Flash) that reads the conversation, decides what to ask or do next, and calls tools.

- a **tool layer** exposed through a small, fixed contract (the Model Context Protocol, MCP): tools to list topics, list the contact media valid for a topic, check availability in a datetime window, book a slot, and read the current time. The tools are deterministic and back onto a reproducible mock booking service.
- an **orchestration layer** (LangGraph) that runs the agent-and-tools loop.

All four layers run together as a working, deployed chatbot. A live demo is available at https://huggingface.co/spaces/JosefPilter/appointments-chatbot, and the full source code with the evaluation artifacts is at https://github.com/RomanPfeiler/appointment-scheduling-chatbot.

In the **baseline**, the agent does all of the language understanding itself, inside one prompt. It reads "next week Tuesday afternoon", decides what window that means, and calls the availability tool. The research question is whether a dedicated **NLP annotation layer** makes the agent measurably better. This layer runs before the agent on each user message, extracts the datetime, the topic, and the contact medium, and hands the agent a structured annotation hint. The study follows a classic baseline → measure → improve → re-measure design: build the simplest thing that works, measure it, add the candidate improvement behind a switch, and re-measure on the same conversations so the difference is attributable.

The NLP layer is measured in two ways, because they answer different questions. The intrinsic view scores the parser and the entity extractor on their own, against a gold set of labelled utterances. Does the component recognise the time and the topic correctly, in isolation? The extrinsic view scores the whole agent on the scenario suite. Does turning the component on change what the agent actually does? The full metric set and the argument for why this is a fair test are laid out in section 2. Throughout, every comparison is paired (the same scenario run under each condition) and statistically tested. No difference is reported as a bare "it got better".

1.4 Scope and non-goals

This is a deliberately bounded study. The system books a single appointment and operates in English only (including the non-native variants described in section 2). The LLM-as-judge uses a single model, scored once per conversation, which limits what the judge can support (see section 4.7). Multilingual evaluation, a cross-model or human-validated judge, and automatic agentic-failure-mode detectors are out of scope. These bounds were fixed in advance so the experiment could give a clean answer, including the answer "the NLP layer does not help".

1.5 Related work

The Schema-Guided Dialogue dataset (SGD) and MultiWOZ supply the mined time-expression phrasings behind the scenarios (section 2). Full citations are in section 6.

2. Evaluation

How do you tell whether one version of a chatbot is better than another? This section sets up the answer the rest of the report depends on. It has three parts: the scenario suite and the language data it is built from (2.1), the metrics each conversation is scored with (2.2), and the argument for why this setup is a fair, informative test (2.3).

2.1 The scenario suite and its data

The unit of evaluation is a **scenario**: one self-contained test of the agent, consisting of a goal (book Investing, online, for a particular time), a **persona** that drives a simulated user (cooperative, negotiating, or adversarial), a fixed opening phrasing drawn from a phrase bank, a controlled calendar, and the expected metrics. The suite holds ∼500 such scenarios, generated combinatorially with a fixed random seed so it regenerates identically.

2.1.1 The phrase bank

The agent is tested on realistic time phrasings, not phrasings invented to flatter the parser. Every scenario's opening request is therefore seeded from a **phrase bank**: user utterances carrying a time expression, mined from two open task-oriented dialogue datasets and tagged by how hard the time expression is to parse.

Sources

The bank draws on the **Schema-Guided Dialogue** dataset (SGD, DSTC8) as the primary source, because it contains banking, calendar, and appointment services, and on **MultiWOZ 2.2** as a secondary source for booking-style negotiation phrasings. English user turns annotated with a date- or time-typed slot were mined, then lightly stripped of domain-specific nouns so a phrasing like "a table for Friday at 7" transfers cleanly to a banking appointment.

Difficulty buckets

Each phrasing is tagged into a **difficulty bucket** that describes how hard its time expression is to parse:

Difficulty bucket	Count	What it is
unsupported	2,861	out of scope (recurring, multi-attendee, …)
specific date	1,750	fully pinned (“14 March at 14:00”)
relative	361	relative to today (“next Tuesday”)
vague	345	under-specified (“after work”)
deadline-driven	38	one-sided bound (“before Friday”)
compound	30	several options (“Tue or Thu afternoon”)
open-ended	14	no time expression at all
negative constraint	5	exclusion (“not Monday”)
Total tagged	5,404	

The distribution is steeply skewed. Out-of-scope requests and fully specified dates are ∼85% of the bank, while the buckets that genuinely stress a parser (vague, compound, deadline-driven, open-ended, negative constraint) are few. Most real requests are easy ones, so a component that helps only on the rare hard requests barely moves the overall averages (section 4).

Source coverage

MultiWOZ contributes ∼3,100 phrasings and SGD ∼2,300, but the useful coverage differs by source. SGD supplies the appointment- and banking-flavoured phrasings, while MultiWOZ supplies the negotiation-style phrasing (“anything that day?”) the booking tiers lean on.

2.1.2 The Swiss-variant addendum

A Swiss bank’s customers include many non-native English speakers whose English carries habits from their first language, whether German, French, or Italian: date order written “14.03”, times like “14 o’clock” or “14h00”, weekday phrasings like “next week Tuesday”. The mined SGD/MultiWOZ phrasings are native American/British English and under-represent these patterns, so a small AI-curated sub-bank of ∼100 entries was added: generated with an LLM assistant instructed to produce each first language’s interference patterns, and reviewed by the author. The sub-bank spans three variants, each covering the main difficulty buckets:

Variant	Count	Example phrasing
en_de (German-influenced)	40	“next week Tuesday at 14 o’clock”, “the 14.03 at half 14”
en_fr (French-influenced)	35	“the 14 of March at 14h00”, “a rendez-vous each week”
en_it (Italian-influenced)	25	“the 14 marzo at 14:30”, “next martedì at 10”

2.1.3 The scenario suite

Scenarios are organised into **seven tiers** of increasing negotiation difficulty. Tiers 1–5 are booking tasks of rising hardness. Tier 6 collects request shapes that do not fit the ladder (deadline-driven, negative-constraint, and open-ended requests). Tier 7 is out-of-scope requests where the correct outcome is a refusal. A deliberate minority of scenarios in Tiers 1–5 are **unreachable** (the calendar is empty in the requested window), so that “no booking” is sometimes the right answer and the booking metric is not a near-constant 100%.

Evaluation example — one scenario per tier

- **Tier 1 (direct match):** the requested time is free, so the correct path is a quick confirm-and-book.
- **Tier 2 (near miss):** the requested day has openings but not the requested time, so the agent must offer nearby times on the same day.
- **Tier 3 (day shift):** the requested day is completely full, so the only correct path to a booking is to recognise this and offer another day.
- **Tier 4 (broad / vague window):** the user names only a loose window (“sometime next week”), so the agent must narrow it down to one concrete slot.
- **Tier 5 (multi-round negotiation):** an adversarial user keeps changing the constraints across several turns, and the agent must hold the context and still converge on a slot.
- **Tier 6 (other):** a one-sided or exclusion request (“anytime before Friday”, “not Monday”), where the agent must respect the constraint rather than book the literal time.
- **Tier 7 (out of scope):** the user asks for something the system cannot do, such as “book me a recurring monthly review”, where the only correct outcome is a clear refusal.

Tier	Description (negotiation difficulty)	Count	Reachable	Unreachable
1	Direct match (requested time is free)	80	72	8
2	Near miss (day free, not the time)	80	72	8
3	Day shift (requested day full)	80	72	8
4	Broad / vague window	80	72	8
5	Multi-round negotiation (adversarial)	80	72	8
6	Other (deadline / negative / open-ended)	60	60	0
7	Out of scope (correct answer is refusal)	40	0	40
Total		500	420	80

Each scenario carries a **language-variant** label from `{en_native, en_de, en_fr, en_it}`, drawn from the phrase bank and its Swiss-variant sub-bank. Non-native phrasing makes up 37% of the 500 scenarios (20% German-influenced, 11% French, 6% Italian), so the evaluation can ask whether parsing degrades on non-native phrasing. It does (see section 3.4).

What is actually run. The ~500 scenarios are the library each experiment draws from, not the number executed in any one run: testing all of them under every condition, repetition, and experiment would cost thousands of conversations. Each reported experiment therefore uses a fixed, seed-frozen selection of **60 scenarios run twice each — 120 conversations per condition**, with every condition evaluated on the identical selection so the comparison stays paired (section 4).

Phrasing–calendar alignment

For the tiers where datetime-parse accuracy matters most (1, 2, 3, 5), the generator parses each opening phrasing and aligns the scenario's calendar so the requested slot is genuinely present (or genuinely absent, for unreachable cases).

2.1.4 The mock booking service

The agent's tools front a **mock booking service**. This is what makes evaluation reproducible.

The service does not store a fixed calendar. It generates availability on demand from a seed derived from the date and the topic, so the same (date, topic) always yields the same slots and a scenario behaves identically every run. Availability is generated relative to today and respects the business rules (24-hour lead time, 08:00–17:00 business hours, 60-minute slots, weekdays only), so the data never goes stale and never offers an illegal slot.

Per-scenario overrides and relative dates

Each scenario can **override** the generated calendar for its run, injecting exactly the availability its tier contract requires (Tier-1 slot free, Tier-3 day full). Scenarios specify dates relative to "today" and resolve both the calendar and the "next Tuesday" phrasing against one pinned reference date per scenario, so the suite keeps testing the same thing on any run.

2.1.5 Limitations of the data

- **Swiss-variant sample size and validity.** The ~100 AI-curated non-native entries are small next to the ~5,400 mined ones. These phrasings reflect what an LLM, guided and reviewed by the author, produces as plausible first-language interference, not a corpus study of how Swiss bank customers actually write English.
- **Parse accuracy is not defined for every scenario.** The metric needs a scenario whose calendar was aligned to its opening phrasing. For about a third of the alignment-critical scenarios the generator could not do this and fell back to a random offset. Those scenarios carry no parse-accuracy score.
- **English only**, and **synthetic, not collected**. The suite is a controlled stand-in for real traffic, not a sample of it (revisited in section 4.7).

2.2 Metrics

Every conversation is scored on a mix of **deterministic** metrics (computed by code from the transcript and tool calls) and an **LLM-as-a-judge** rating (scoring conversation quality on a rubric). The deterministic ones are the backbone. The judge adds the qualitative dimensions the deterministic ones miss.

Metric	What it measures	Why it matters here
Booking completion	reached the correct outcome: a real booking, or a correct "nothing available"/refusal	the headline task-success metric
Turn count	user ↔ agent exchanges to resolution (12-turn cap)	user effort

Metric	What it measures	Why it matters here
Dead-end turns	an availability check returned empty and the user saw no slots	the "nothing available" wall, a frustration proxy
Parse accuracy	first availability window vs the gold window (1.0 right day+time, 0.5 right day, 0 wrong day)	does the agent act on the right time
Faithfulness	every slot, topic, contact medium, or booking the agent states must come from an actual tool result, never one the agent made up	catches fabrication
Call efficiency + latency	availability calls vs the theoretical optimum, plus simulated wait time	wasted work and user wait
Judge: five dimensions (LLM)	Temporal, Negotiation, Efficiency, Customer Experience, Recovery, each 1–5	qualitative quality the code-level metrics cannot see

Every comparison in this report is **paired** (the same scenario is run under each condition), and reported as an effect size with a 95% confidence interval. The judge dimensions and the deterministic metrics are designed to cross-check, not duplicate, each other (a lesson that pays off in section 4).

2.3 Why this is a valid way to assess the chatbot

A good evaluation has to be hard to game and easy to attribute. Six design choices make this one so:

1. **The seven tiers stress different skills.** Direct match tests recognition. Near-miss and day-shift test whether the agent proposes alternatives. Broad-window tests how it chunks a vague request. Multi-round tests context retention under pushback. Out-of-scope tests refusal.
2. **"No booking" is sometimes the correct answer.** Because ~16% of the suite is deliberately unreachable (the empty-calendar minority in Tiers 1–5 plus the whole out-of-scope Tier 7), a chatbot that fabricates a booking to look successful is penalised, and the booking metric stays discriminating instead of pinned near 100%.
3. **Deterministic metrics anchor the subjective ones.** Booking, parse accuracy, faithfulness, and dead-ends are computed by code with no model in the loop, so they are reproducible and immune to a judge's variability. They are the cross-check that caught a misleading judge result in section 4.
4. **The paired design makes differences attributable.** Running the identical scenario under each condition removes scenario-mix as a confound. A measured difference is the condition, not luck of the draw.
5. **A persona-driven user simulator supplies realistic pressure.** Cooperative, negotiating, and adversarial personas push back, change their minds, and (on Tier 7) hold an out-of-scope request, while a frozen opening phrasing fixes the exact input the parser sees, so parse accuracy is comparable across conditions.

3. NLP Analysis

This section is about the candidate improvement, the **NLP annotation layer** that runs before the agent on each user message. It describes what the layer does, the three conditions under comparison, and how the components score in isolation (intrinsic evaluation). Whether any of it changes the agent's behaviour is the extrinsic story, previewed at the end and settled in section 4.

3.1 What the annotation layer does

In the baseline, the agent reads the raw user message and does all the language work itself. The annotation layer inserts one step before the agent. On each new user message it runs two small NLP components and hands the agent a structured annotation hint.

- A **datetime parser** finds the time expression, normalises it, and tags it with a **specificity**: an exact time ("Tuesday at 14:00"), a specific day, a vague day ("after work"), a multi-day window, or a compound ("Tuesday or Thursday"). The specificity signals how wide a window the agent should check. A pinned time wants a tight one-hour query, a vague "next week" the three-day maximum. It targets a window-sizing inefficiency the baseline shows. After an empty availability check, the agent re-queries the same narrow window instead of broadening (section 4.1).
- An **entity extractor** pulls the topic (Investing / Mortgage / Pension) and the contact medium (branch / online / phone) out of the message, so the agent can skip a "what would you like to discuss?" round-trip when the opener already says it.

The annotation hint is injected into the agent's prompt as a short structured block.

The example below shows the layer's output on a real opening message from the scenario suite (a Tier-1, native-English run with the API-LLM pipeline).

> **Evaluation example — annotation-layer output (Tier 1, native English)**
>
> **User message:** "Hello, my name is Alex Smith. I'd like to book an online meeting about pension. Anything available at afternoon 2:15 tomorrow?"
>
> **Structured annotation hint handed to the agent:** topic = `pension`, contact medium = `online`, datetime = "tomorrow at 14:15" resolved to 2026-06-02 14:15–15:15, tagged specificity `exact_time`.
>
> The parser normalises "afternoon 2:15 tomorrow" into a one-hour window and tags it `exact_time`, which tells the agent to check a tight window. The entity extractor fills in the topic and contact medium the opener already states. With the layer switched off, the agent receives only the raw message and has to derive all of this itself.

A Tier-4 run from the latest API-LLM evaluation shows the layer on a vaguer, more open-ended booking request, where the day is given but the exact time is not.

> **Evaluation example — annotation-layer output (Tier 4, broad / vague window)**
>
> **User message (opener, negotiating persona):** "Hello, my name is Alex Smith. I'd like to book a phone meeting to discuss investment options. That sounds great. I want to schedule a visit on Thursday next week."
>
> **Structured annotation hint handed to the agent:** topic = `investing`, contact medium = `phone`, datetime = "next Thursday" resolved to that whole working day (00:00–23:59), tagged specificity `day_vague`.
>
> Tier-4 openers name a day or a rough region of time, but not an exact slot. Here the parser pins the day ("next Thursday") yet, because no time of day is given, tags it `day_vague` and widens the window to the whole day rather than a single hour — exactly the signal the agent needs to offer slots across that day instead of probing one time. The entity extractor fills in the topic and contact medium in the same pass.

3.2 The three conditions

The Gemini-only **baseline** is compared against three NLP conditions. They differ in the mechanism that does the recognition, but all are given the same **canonical synonym knowledge** — one hand-curated list that maps surface words to a fixed set of labels (e.g. "portfolio", "stocks", and "pillar 3a" all map to the topic *Investing*; "call me" and "by phone" map to the contact medium *phone*) — and normalise to the same structured output, so the comparison is "different mechanisms over the same knowledge", not "one mechanism with a richer vocabulary".

Condition	Datetime parser	Entity extractor
Classical pipeline	dateparser library + surface-text rules to assign specificity	spaCy phrase-matcher over the canonical synonym list
Local-LLM pipeline	on-premises quantised Llama-3.2-3B	same local model
API-LLM pipeline	Gemini 2.5 Flash in JSON mode	same API model, with the synonyms in the prompt

The two LLM conditions differ only in how they produce the structure. The local model annotates in place with lightweight grammar-constrained tags (a 3-billion-parameter model wastes too much of its limited capacity on JSON punctuation), while the API model uses strict JSON mode. The local model is **Llama-3.2-3B**, chosen over a same-size Qwen2.5-3B in a small pilot on the project's own gold set (Llama alone broke the German-variant vague wall and false-fired far less on the hard-negative controls).

3.3 Intrinsic evaluation: scoring the components on their own

Extrinsic metrics show whether the agent changed. They cannot show whether the component is any good, because the agent can quietly correct a bad annotation hint or ignore a good one. So each component is also scored directly, on independent gold sets of labelled utterances: 150 for datetime, 200 for entities.

This entity (topic and contact medium) gold set is sourced independently and deliberately includes paraphrases that are not in the list ("I want to put some money to work" for Investing, "can someone ring me?" for phone). The datetime gold set is deliberately spread across the difficulty buckets and the four language variants, so it can expose where non-native phrasing breaks a parser.

3.4 Intrinsic results

3.4.1 Datetime: recognition rises, and the classical pipeline struggles with non native phrasing

The two columns below measure different things. **Parse rate** is how often the component finds and normalises the time expression at all. **Specificity** is, of the expressions it *did* parse, how often it then assigned a sensible specificity tag. The parser can label each time as an exact time, a specific day, a vague day, a multi-day window, or a compound, and that tag tells the agent how wide a window to check. A high parse rate paired with a low specificity score would mean the component reads the time but mis-judges how precise it is (for example tagging "14:00" as a vague window), which would mislead the agent's window sizing. Overall, the API-LLM pipeline recognises the most time expressions, and both LLM pipelines are far more consistent in the specificity they assign:

Condition	Parse rate (overall)	Specificity
Classical pipeline	78.7 %	77.3 %
Local-LLM pipeline	78.7 %	96.7 %
API-LLM pipeline	84.0 %	95.5 %

The overall parse rate hides the interesting structure, which is by language variant (Figure 2). The classical pipeline handles native English well but hits a wall on non-native phrasing: it scores 0% on every non-native vague cell, and only 75–89% (against ~100% for the LLM pipelines) on the non-native specific-date and relative cells. It simply does not understand "the 14.03 at half 14" or "next week Tuesday" correctly. The API-LLM performs the best. A rule-based parser encodes the date formats its authors anticipated. A multilingual LLM has seen these phrasing patterns in its training data.

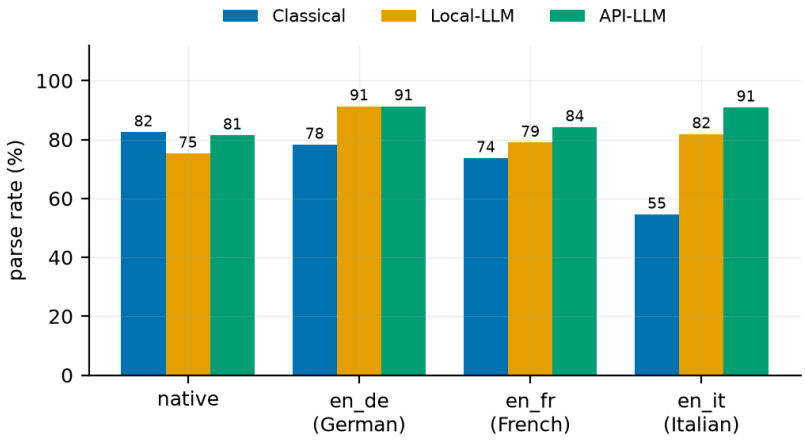


Figure 2. Intrinsic datetime parse rate by condition and language variant (all difficulty buckets pooled per variant). The classical pipeline drops sharply on non-native variants (the "Swiss-variant wall", worst on Italian). Both LLM pipelines hold up, the API-LLM pipeline best.

3.4.2 Entities: the paraphrase wall

The headline entity result is a clean split (Figure 3), and the **F1 score** captures it in a single number. F1 is the standard accuracy measure for this kind of recognition. It is high only when the component both finds the entities that are there and does not invent ones that are not, so it falls to near-zero when a component simply fails to recognise an entity. On **canonical** phrasings, where the user names the entity outright ("investing", "phone") — all three conditions are effectively perfect. On **paraphrases**, phrasings not in the synonym list, like "I want to put some money to work" the classical phrase-matcher collapses to F1 ≈ 0, while the LLM pipelines stay strong.

Utterance kind	Classical (spaCy)	Local-LLM	API-LLM
canonical (entity named outright)	1.00	0.99	1.00
paraphrase (entity only implied)	0.00	0.88	0.99

Entity F1, macro-averaged over the three topics and three contact media.

The reason is in how the classical matcher works. It is, in effect, a dictionary lookup. spaCy serves only as a fast tokeniser feeding a phrase matcher that scans each message for the hand-authored synonyms (investing = invest / portfolio / stocks / pillar 3a / …). It recognises the words it was given, not the meaning, so a paraphrase that uses none of those words is invisible to it by construction.

What it means for the project

Reading paraphrases is the one place the LLM pipelines clearly beat the dictionary matcher, and the API-LLM pipeline does it almost perfectly. The local model sits sensibly between the two: far better than the matcher on paraphrases, a step below the API model.

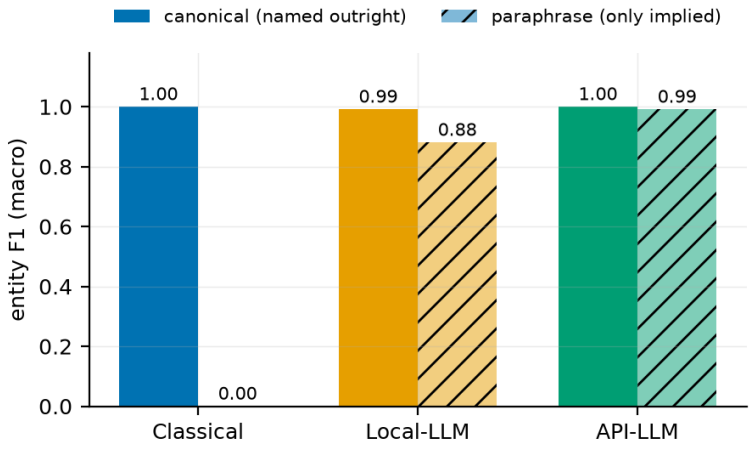


Figure 3. Entity F1 by condition, on canonical phrasings (entity named outright) versus paraphrases (entity only implied, not in the synonym list). All three are near-perfect on canonical phrasings; on paraphrases the classical dictionary matcher collapses to F1 ≈ 0, while the LLM pipelines hold up (local-LLM 0.88, API-LLM 0.99).

3.5 From intrinsic to extrinsic

The intrinsic results characterise the three NLP approaches against each other, not against the baseline. The Gemini-only baseline has no standalone parser to score. It does its date reading silently, inside the agent.

The extrinsic measure is computed from the agent's behaviour, not the component's. For each conversation it takes the window of the agent's **first availability tool call** and compares it to the scenario's gold window, scoring 1.0 for the right day *and* time, 0.5 for the right day only, and 0 for the wrong day, then averages across the evaluated conversations (section 2.2). It therefore asks "did the agent act on the right time?", not "did the parser read the right time?".

Measured this way, all three NLP conditions sit only marginally and non-significantly above the baseline (0.41): classical 0.43, local-LLM 0.48, API-LLM 0.46 (Figure 4). And no condition significantly moves booking, turns, or faithfulness. The recognition differences stay trapped at the component layer. Why none of this converts, and one intervention that did move the agent, is the subject of section 4.

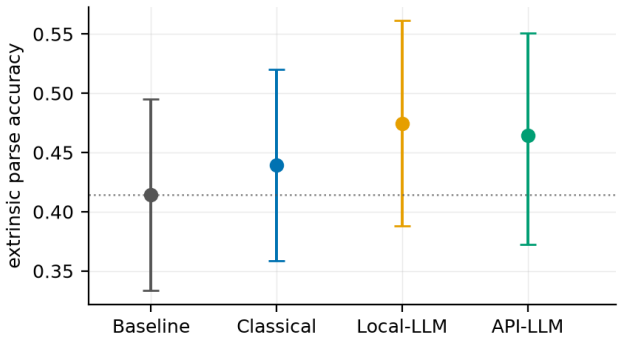


Figure 4. Extrinsic datetime-parse accuracy by condition, with 95% confidence intervals. All three NLP conditions sit marginally above the baseline (Δ +0.02…+0.07) but none significantly — every interval comfortably includes zero — the first sign that recognition gains do not convert into agent behaviour.

4. Results & Discussion

All headline numbers below come from paired runs of 120 conversations per condition on an identical scenario selection, tested with the paired statistics described in section 2.2. Three experiments are reported: the **NLP layer** (baseline vs the three conditions), the **window-expansion policy** (a deterministic executor change, NLP off), and their **combination**.

4.1 How the Gemini-only baseline behaves

The unaided baseline has one dominant failure mode that shapes every result that follows. The table below describes it on the representative baseline run with 60 scenarios × 2 repetitions of the Gemini-only agent (120 conversations). This is the baseline the NLP comparisons in section 4.2 are paired against (Figure 5):

Tier	Median turns	Average turns	Booking
1	3.0	5.2	70.0 %
2	7.5	7.5	50.0 %
3	5.0	7.1	65.0 %
4	6.0	6.4	90.0 %
5	8.5	8.7	80.0 %
6	4.5	6.0	80.0 %
7	4.0	3.7	0 %*
Overall	**5.0**	**6.4**	**62.5 %**

*Tier 7 is out-of-scope, so the correct outcome is a refusal: 0% booked means the agent correctly declined every time.

Two things stand out. First, a capable base model needs little hand-holding on a clean request. The median sits low on the easy tiers. Second, the turn counts fall into two separate groups, and the gap between the **median** and the higher **average** is the tell-tale sign — a cluster of fast successes pulls the median down, while a tail of failures running all the way to the 12-turn cap pulls the average up (clearest on Tiers 1 and 3, where the average is about two turns above the median). Reading the long-running transcripts revealed the dominant failure mode, which is **window-sizing inefficiency**. When a narrow availability check comes back empty, the agent re-queries the same narrow window instead of broadening, and the customer experiences a wall of "sorry, nothing available".

> **Evaluation example — Tier 2, window-sizing failure (baseline)**
>
> A cooperative customer asks for a 16:00 slot. The agent issues eleven successive one-hour checks at exactly 16:00 across eleven different weekdays, never widening the window, until it hits the turn cap with no booking.
>
> **Judge (verbatim, sibling case):** "The conversation was stuck in a loop, with the agent repeatedly checking for the same unavailable 9 AM slot across multiple days without proactively suggesting different times."

This is not a recognition failure (the agent understood the time perfectly) but a planning failure in how it uses the availability tool. It was made a first-class deterministic metric (the **dead-end turn**) and became the target of the project's one clear end-to-end win (section 4.4).

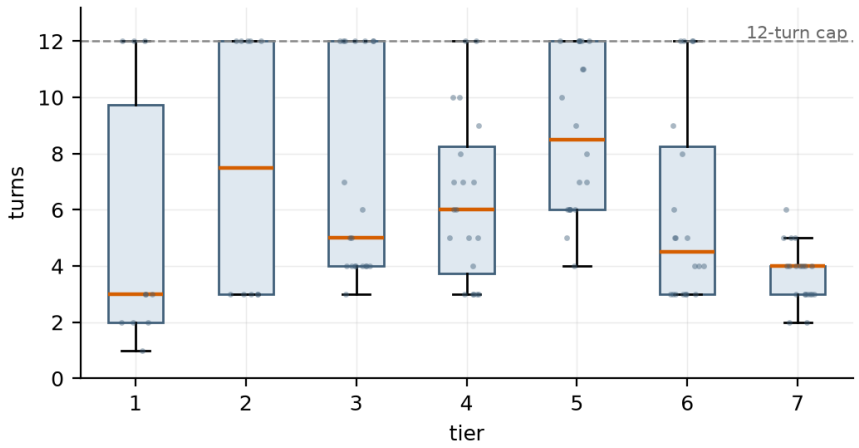


Figure 5. Baseline turn-count distribution per tier (the Gemini-only agent, representative baseline run, 120 conversations); failures pile up at the 12-turn cap.

4.2 Datetime recognition improves slightly, while the agent does not

Structured NLP improves datetime recognition slightly (non-significant) but does not improve the agent end-to-end

The main findings here are:

- The entity and datetime parsing is improving slightly (although statistically not significant) from the baseline across the NLP conditions when measured extrinsically as seen in section 3.5 and figure 4.
- Despite that nothing converts across booking, turn count, and dead-end turns. No NLP condition produces a significant improvement over the baseline.

The four conditions, side by side:

Condition	Booked	Median turns	Faithfulness	Average Dead-end turns	Extrinsic parse
Baseline (Gemini only)	62.5 %	5.0	95 %	3.1	0.41
Classical pipeline	63.3 %	5.0	94 %	3.1	0.43
Local-LLM pipeline	54.2 %	6.0	96 %	3.4	0.48
API-LLM pipeline	59.2 %	6.0	94 %	3.2	0.46

4.3 The results in detail

4.3.1 No improvement on booking completion

Booking is the headline KPI, scored as "reached the correct outcome", and it is down across the NLP conditions (Figure 6). The sample here is fairly small (120 paired conversations), so this dip may not be a reliable difference. The overlapping confidence intervals in Figure 6 reflect that.

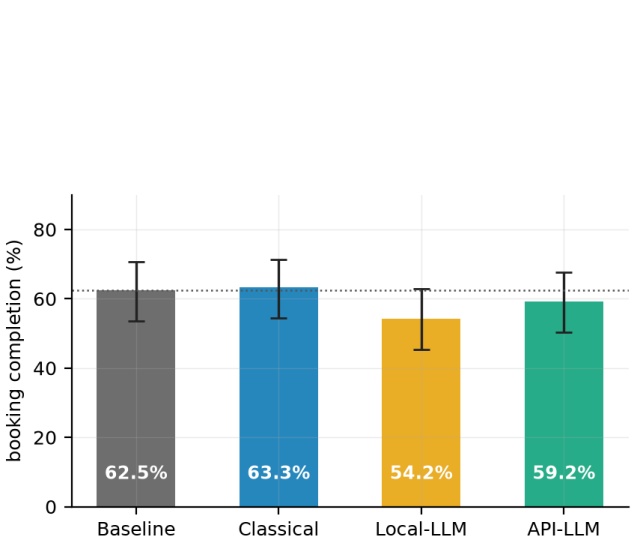
The one condition that lands on the baseline is the classical pipeline (63.3% vs 62.5%). The classical arm matches the baseline because it changes the agent the least. Its annotation barely shifts the agent's behaviour. The two LLM pipelines parse better in isolation yet book *worse* (local-LLM 54.2%, API-LLM 59.2%), because their more confident, narrower windows and their occasional broken mid-conversation hints (section 4.3.3) actually reach the tools.

4.3.2 Turn economy, judge dimensions, and faithfulness

Turn count and dead-end turns are flat for the classical and API-LLM pipelines. The **local-LLM pipeline is the one condition that significantly degrades the agent**. It adds turns (+0.79) and makes the agent waste more availability checks. Here, call efficiency means how many availability checks the agent makes compared to the fewest it would need, so a lower number is better. Under the local-LLM pipeline that number rises significantly, once two runaway scenarios that hit the turn limit are set aside. The annotation hints help on the precise tier, where the opener already pins the exact time (Tier 1, efficiency 5.1 → 2.2, booking 70% → 90%), but hurt on the negotiation tiers, where success requires widening the search (Tier 2, efficiency 6.3 → 18.3, booking 50% → 30%). A confident-but-narrow first window, fed to an agent that does not broaden on its own, produces an empty result and a retry loop.

The judge corroborates (Figure 7). The classical and API-LLM pipelines have similar results to the baseline, while the local-LLM pipeline sits significantly below it on negotiation (−0.45), efficiency (−0.38), and recovery (−0.42).

Faithfulness, finally, is high and statistically flat everywhere (94–96%).



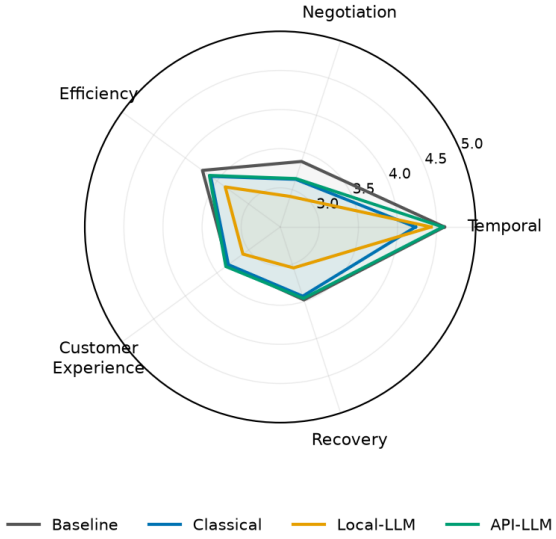


Figure 6 (left). Booking completion by condition with 95% confidence intervals. **Figure 7 (right).** Judge radar. The classical and API-LLM pipelines hug the baseline. The local-LLM pipeline is pulled in on negotiation, efficiency, and recovery.

4.3.3 Why the annotation hints don't convert: the mechanism

To find the issue, I looked at the conversations where the booking outcome changed between the baseline and the NLP condition, at the first turn where the two versions started to differ, and at the exact annotation hint the agent was given on that turn. Two properties of the annotation hint itself turn out to do the damage.

The annotation hint narrows the first search

The baseline automatically queries a larger time window. For example when asked for "Monday at 8", it typically checks a window of a day or more and catches the 11:00 slot the literal request would have missed. The annotation hint's window guidance ("exact time → check a tight one-hour window") overrides that habit.

Mid-conversation annotation hints are systematically wrong

The annotation layer re-parses every user message in isolation, with no conversation context. Turn-1 openers parse well (the small turn-1 parse gain that does show up extrinsically), but follow-ups are short phrases that point back to something said earlier ("something closer to 9 AM on that Thursday"), and 14–34% of mid-conversation annotation hints come out badly broken, such as midnight windows, wrong days, even wrong years. This broken output demonstrably reaches the tools. Availability checks starting at midnight, which essentially never occur unaided, rise from 2 baseline calls to 72 / 373 / 183 under the three pipelines.

The payoff is therefore asymmetric

Where the turn-1 annotation hint was day-correct, conversations book at roughly the baseline rate. Where it was wrong and the agent adopted it, bookings collapse.

> **Evaluation example — wrong-and-adopted annotation hint (Swiss-format date, API-LLM pipeline)**
>
> The opener carried a Swiss-format date, "around the 14.03."
>
> **Baseline:** noticed the date was implausible and asked the customer to confirm, booking in three turns.
>
> **With the annotation hint:** the agent silently queried the resolver's confident-but-wrong March-2027 reading and stayed on it for nine turns.

This is documented LLM behaviour, where models adopt coherent, authoritative-looking external evidence even when it is wrong (Xie et al., 2024), and irrelevant added context degrades problem-solving even when models are told to ignore it (Shi et al., 2023). The annotation hint block is exactly that kind of context. How this plays out on a weaker base model is taken up in section 4.6.

4.4 The window-expansion policy: the one significant end-to-end win

The NLP null pointed at where the agent actually loses: not in recognition, but in what it does after an empty availability check, the narrow-probe, never-broaden loop of section 4.1. So the next experiment moved that one decision out of the LLM's discretion and into deterministic code. The **window-expansion policy**: when an availability check comes back empty, the executor automatically runs a bidirectional widening search (the requested day, then a few days forward, then a few days back) and surfaces the nearest real slots in the same turn. It is a pure executor change, tested with the NLP layer off, so the comparison isolates "agentic strategy" from "NLP".

This is **the first intervention in the project to move end-to-end agent quality significantly** (Figures 8–9):

Metric	Baseline	+ Expansion	Δ	95% CI of Δ
Average Dead-end turns	3.08	1.42	−1.66	[−2.30, −1.00]
Turn count	6.32	5.69	−0.63	[−1.20, −0.06]
Judge: Negotiation	3.36	3.78	+0.43	[+0.11, +0.72]
Judge: Customer Experience	3.34	3.73	+0.38	[+0.12, +0.66]
Booked	63.0 %	65.5 %	+2.5 pp	[−4.2, +9.2] pp
Simulated latency / conv	10.0 s	13.0 s	+2.95 s	the cost (see text)

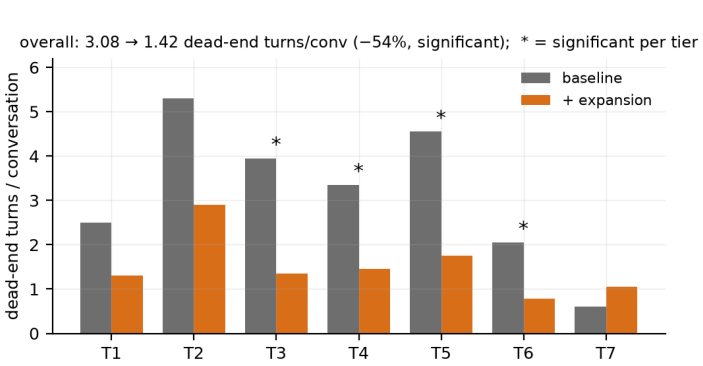
Read the confidence intervals, not the point estimates. Four of the five exclude zero, so those effects are real and unlikely to be chance: the cut in dead-end turns, the shorter conversations, and both judge gains. Booking's interval includes zero, so that change is not significant at this sample size. The added latency is the cost of the policy.

The headline is a **54% cut in dead-end turns** (which means a "sorry, nothing available" response in the chat). The mechanism is visible in a single before/after pair on the same scenario:

> **Evaluation example — day-shift scenario, requested day fully booked**
>
> **Baseline:** the agent checks the requested Tuesday, gets nothing, checks the same window on Wednesday, gets nothing, and keeps re-asking the customer to "pick another time" — five dead-end turns before it gives up.
>
> **With expansion:** the first empty check silently triggers the widening search, which finds two real slots two days later, and the agent opens its next message with "Tuesday is full, but I have Thursday at 10:00 or 14:00." One turn, no wall.

The clearest aggregate case is Tier 3 (requested day full, agent must shift days), where dead-ends fall 3.95 → 1.35, the call-efficiency improves (8.65 → 3.62), and booking rises 65% → 80%. The judge corroborates with significant gains on Negotiation and Customer Experience.

The window broadening removes the dead-end wall but does not, at this sample size, significantly increase completions. Additionally, latency rises a measured +2.95 s/conv. This improvement can be considered as variance and outlier improvement, with a latency cost.



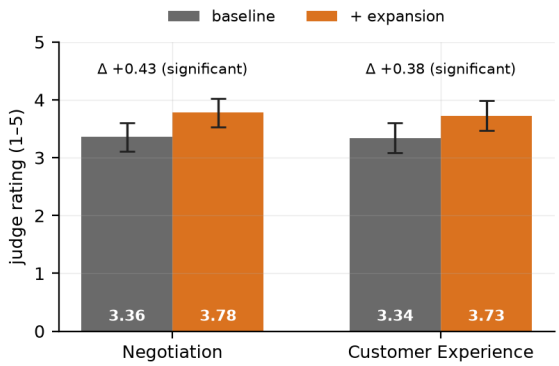


Figure 8 (left). Dead-end turns per conversation, baseline vs window-expansion, per tier: the −54% win, concentrated in the negotiation tiers (3–5); * marks tiers where the reduction is significant. **Figure 9 (right).** Judge Negotiation and Customer-Experience ratings with 95% confidence intervals: both a significant +0.4 (the intervals exclude zero).

4.5 Combining the two interventions: a redundancy result

The two levers act on the same failure mode (the empty-window loop) from opposite ends. If they are **complementary**, better upfront parsing (the API-LLM pipeline) makes the first availability check correct more often, so the combination should beat window expansion alone. If they are **redundant**, the window expansion already rescues the over-narrow windows on its own, and better parsing has nothing left to fix.

The combination (the API-LLM pipeline and the expansion policy) was run and paired against expansion-only.

Metric (combination − expansion-only)	Δ	Significance
Dead-end turns / conv	−0.19	n.s.
Turn count	−0.31	n.s.
Booking	+0.8 pp	n.s.
Faithfulness	+1.7 pp	n.s.
All five judge dimensions	≤ 0.08	n.s.

The metrics lean slightly towards the combined approach, although all of them are non-significant improvement. The NLP layer does not seem to improve the results whether window expansion is on or off, while the expansion policy's wins replicate whether NLP is on or off.

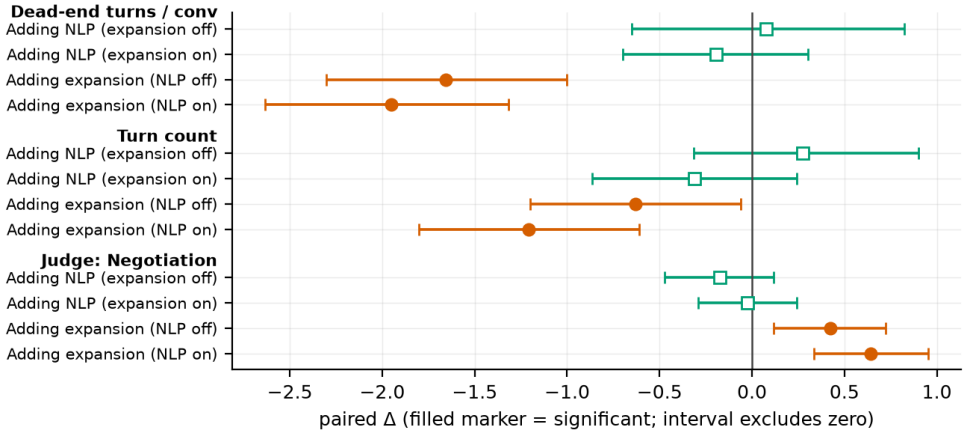


Figure 10. Does each lever still work when the other is already present? The paired effects are grouped by metric (dead-end turns, turn count, judge negotiation). Within each group, the first two rows are the effect of switching the NLP layer **on** — once with window expansion off, once with it on — and the next two are the effect of switching window expansion **on** — once with the NLP layer off, once with it on. Each point is a paired effect with its 95% confidence interval (a filled marker means the interval excludes zero, i.e. a real effect). The NLP layer changes nothing in either context; window expansion helps in both. The two levers are therefore redundant rather than additive: expansion delivers the same win with or without the NLP layer.

4.6 Discussion

Why the NLP layer does not convert

The mechanism is in section 4.3.3. The base model already uses the windows, the annotation hints would supply and a wrong annotation hint confuses the base model. On top of that, the NLP layer's one advantage over the other pipelines (reading non-native phrasings, catching paraphrases) shows up only intrinsically, covers a small slice of the test data, and is something the baseline's own API model already handles. It has little extra work to do in front of a capable planner, and is costly when it is wrong.

Why the executor change does convert

The agent's real, frequent loss is not recognition but strategy after an empty result, and that is the decision a deterministic policy can make better. The combination then shows the deeper point. It seems that the deterministic policy absorbs a failure mode, and the recognition improvement aimed at the same failure mode has nothing left to fix.

Does this result depend on the agent being strong? A weak-agent experiment

Perhaps the annotation hints are redundant only because Gemini 2.5 Flash is a strong in-context planner, and a weaker agent has the headroom the annotation hints could fill. A smoke-scale probe (14 scenarios, one repetition: directional evidence only) swapped the agent to `gemini-2.5-flash-lite` (thinking disabled), every other component was the same. The weak agent has real headroom (booking 50% vs the strong baseline's ~64% on the same scenarios).

Extrinsic parse accuracy rises steadily with annotation hint quality (0.38 → 0.46–0.63). **Yet booking completion decreases under every NLP condition** (50% → 43% / 36% / 36%), and the deterministic expansion policy replicates its win (+23 points booking directionally, with dead-end turns −2.15 per conversation). The section 4.3.3 mechanism explains the contrast. The strong model recovers from wrong annotation hints by re-checking, paying in calls and latency. The weak model does not do that loop, so the same wrong inputs surface as late-conversation failures. The same query-narrowing pattern appears in both families:

First availability check ≤ 1 hour wide	baseline	classical	local-LLM	API-LLM
Strong agent (120 conv/condition)	29 %	31 %	47 %	56 %
Weak agent (smoke, 14 conv, directional)	17 %	54 %	67 %	50 %

The best configuration

With the current setup NLP layer adds no end-to-end benefit on the reliable metrics, in any of its three conditions, so there is no measured case for enabling it by default. The window-expansion policy is the one lever that moves customer-experience metrics. Further experimentation in finding better window expansion strategies could theoretically preserve the NLP wins.

4.7 Limitations

- **Single LLM judge, one repetition.** Judge dimensions are used only as corroboration of deterministic metrics.
- **Extrinsic NLP measurement.** The NLP layer is measured through downstream agent behaviour, so a low parse-accuracy number can reflect agent planning rather than component failure.
- **Synthetic scenarios, English only** (section 2.1.5).
- **Booking completion results are statistically non-significant**, with 5–8 pp effects.
- **Two base models, unevenly powered.** The main study uses one agent model. The weak-agent probe (section 4.6) is smoke-scale.

5. Conclusion & Outlook

5.1 Conclusion

The project asked whether a structured NLP layer makes a tool-calling appointment-booking agent measurably better than a single-prompt baseline. Comparing each condition against that baseline on the same set of conversations gives a clear answer.

No NLP condition significantly moved booking, turn count, faithfulness, or extrinsic parse accuracy at the agent level. A capable base model already plans, in context, the windows the annotation hints would supply, so a correct annotation hint duplicates what the agent would have done anyway, while a wrong one anchors it to a window it then has to recover from (section 4.3.3). The single significant end-to-end win came from a different kind of change: a **deterministic window-expansion policy** in the executor cut "nothing available" dead-end turns by 54% and lifted the judge's negotiation and customer-experience ratings, at a measured latency cost. Combining the two levers showed them to be redundant rather than complementary — the executor policy already absorbs the failure mode better parsing would target.

The methodological contribution is the part of the project that holds up best. The evaluation is what made the behaviour clear. Three choices did the work. A **paired design** (the same seed-frozen scenarios replayed under every condition) turned otherwise-noisy comparisons into effects with confidence intervals, so a null result (the NLP layer) and a genuine win (window expansion) could be told with this sample size. **Deterministic metrics** computed straight from the transcripts anchored the LLM judge and cross-checked it. And splitting the measurement into an **intrinsic** stage (each NLP component scored on its own gold set) and an **extrinsic** stage (the agent's end-to-end behaviour) localised the central finding, that recognition genuinely improved, but the gain never improved the overall result. None of these conclusions would have been legible from a single end-to-end accuracy number.

The architectural contribution is a cleanly separated system (a deterministic tool layer behind a fixed contract, an orchestrator running the agent-and-tools loop, and an optional NLP annotation layer) in which each piece is independently testable and switchable. That separation is what allowed the same scenarios to run with the NLP layer on and off, and with the executor policy on and off, with the differences cleanly attributable.

5.2 Outlook

- **A powered weak-agent run:** The weak-agent probe is smoke-scale. A full run could give more insights.

- **Multilingual evaluation:** The natural extension is to evaluate in German, French, and Italian, rather than only on non-native English phrasings.
- **The planner/renderer split:** Separating a planner (which produces a structured plan) from a renderer that may only put approved facts into words could improve some indicators further.
- **A latency-aware agent:** Surfacing the latency model to the agent would let it plan deliberately around the window-expansion trade-off.
- **LLM powered window-expansion:** A "smarter" window-expansion strategy could work around the failure modes introduced by the NLP layer.

5.3 Acknowledgements

I want to thank the CAS NLP teaching staff at the University of Bern for the course and project framing.

5.4 AI-use disclosure

AI assistance, primarily Claude Opus 4.6, 4.7, 4.8 (Anthropic) used through the Claude Code coding assistant, was a substantial tool in both the implementation and the writing of this project, under the author's direction and review:

- **Code.** The large majority of the code (the LangGraph agent, the MCP tool server, the mock booking service, the NLP pipelines, the evaluation harness with its scenario generator, metrics, and statistics, and the test suite) was written by the AI assistant from the author's specifications and design documents, in iterative sessions in which the author set the task, reviewed the changes, and accepted or corrected them.
- **Data.** The Swiss-variant sub-bank of non-native phrasings (section 2.1.2) was generated by the AI assistant and reviewed by the author, and the intrinsic gold sets were built the same way: AI-drafted, author-reviewed.
- **Analysis.** Evaluation runs, metrics, and statistics were computed by deterministic, AI-written code. Every number reported here traces to frozen result-snapshot files produced by those runs, not to AI-generated text.
- **Report.** This report was drafted by the AI assistant, from the project's progress logs and frozen result snapshots, and then expanded, rewritten and amended by the author.

The research question and its framing, the evaluation design, the decisions taken at each step, and the interpretation of the results are the author's. The author has reviewed all code, data, and text, and takes responsibility for the contents of this report.

6. References

6.1 References

(Author-year style, with the BibTeX source entries kept in `references.bib`.)

Datasets

- Rastogi, A., Zang, X., Sunkara, S., Gupta, R., and Khaitan, P. (2020). Towards Scalable Multi-Domain Conversational Agents: The Schema-Guided Dialogue Dataset. *Proceedings of the AAAI Conference on Artificial Intelligence.*
- Zang, X., Rastogi, A., Sunkara, S., Gupta, R., Zhang, J., and Chen, J. (2020). MultiWOZ 2.2: A Dialogue Dataset with Additional Annotation Corrections and State Tracking Baselines. *Proceedings of the 2nd Workshop on NLP for Conversational AI (ACL).*
- Budzianowski, P., Wen, T.-H., Tseng, B.-H., Casanueva, I., Ultes, S., Ramadan, O., and Gašić, M. (2018). MultiWOZ – A Large-Scale Multi-Domain Wizard-of-Oz Dataset for Task-Oriented Dialogue Modelling. *Proceedings of EMNLP.*

Context effects (sections 4.3.3, 4.6)

- Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E. H., Schärli, N., and Zhou, D. (2023). Large Language Models Can Be Easily Distracted by Irrelevant Context. *Proceedings of ICML.*
- Xie, J., Zhang, K., Chen, J., Lou, R., and Su, Y. (2024). Adaptive Chameleon or Stubborn Sloth: Revealing the Behavior of Large Language Models in Knowledge Conflicts. *Proceedings of ICLR.*

Models

- Gemini, Google (2024). Gemini: A Family of Highly Capable Multimodal Models. Google DeepMind technical report. (The agent and the API-LLM condition use `gemini-2.5-flash`. The weak-agent probe uses `gemini-2.5-flash-lite`.)

- Grattafiori, A., et al. (Llama Team, Meta AI) (2024). The Llama 3 Herd of Models. *arXiv:2407.21783.* (The local-LLM condition uses Llama-3.2-3B-Instruct, Q4_K_M quantisation.)

Tools and frameworks

- LangChain, Inc. (2024). LangGraph: Building Stateful, Multi-Actor Applications with LLMs. https://github.com/langchain-ai/langgraph
- Anthropic (2024). Model Context Protocol (MCP) Specification. https://modelcontextprotocol.io
- Chainlit (2024). Chainlit: Build Conversational AI Applications. https://github.com/Chainlit/chainlit
- Scrapinghub / Zyte (2024). dateparser: Python parser for human-readable dates. https://github.com/scrapinghub/dateparser
- Honnibal, M., Montani, I., Van Landeghem, S., and Boyd, A. (2020). spaCy: Industrial-Strength Natural Language Processing in Python. https://spacy.io

6.2 Dataset licenses

The phrase bank used in this project is built from SGD and MultiWOZ 2.2:

Dataset (role)	License
SGD, DSTC8 (primary source)	**CC BY-SA 4.0** (upstream repository. The mirror used, `bentrevett/schema_guided_dialog`, carries no tag, so the upstream license governs)
MultiWOZ 2.2 (secondary source)	**MIT** upstream. Mirror `tuetschek/multi_woz_v22` tagged **Apache-2.0**, both permissive